

목차 (Table of Contents)

1. 환경 요구사항
2. 설치 순서
 - 2-1. uv 패키지 매니저 설치
 - 2-2. 레포 클론
 - 2-3. 가상환경 생성
 - 2-4. flash-attn 이슈 해결 (의존성 설치 전에 처리)
 - 2-5. 의존성 설치
 - 2-6. SDPA 어텐션 사용 설정
 - 2-7. HuggingFace 토큰 로그인
 - 2-8. 모델 다운로드
 - 2-9. 데이터셋 다운로드
3. 실행 방법
 - 가상환경 활성화
 - 테스트 추론
4. WSL2 주의사항
 - NVIDIA 드라이버
 - .wslconfig 메모리 설정
 - CUDA_HOME 미설정 시 flash-attn 빌드 실패
5. 알려진 이슈 및 해결법
 - flash-attn 빌드 실패
 - 12GB VRAM에서 OOM (Out of Memory)
6. 참고 링크

Alpamayo-R1-10B 설치 및 실행 가이드

WSL2 + RTX 3080 Ti 환경에서 동작 확인된 내용을 기반으로 작성되었다.

1. 환경 요구사항

항목	버전
OS	WSL2 (Ubuntu)
Python	3.12.x
PyTorch	2.8.0+cu128
CUDA (드라이버)	12.6 (nvidia-smi 기준. PyTorch는 cu128 빌드 사용)
GPU	RTX 3080 Ti (12GB VRAM)

2. 설치 순서

2-1. uv 패키지 매니저 설치

```
curl -Lsf https://astral.sh/uv/install.sh | sh
source ~/.bashrc
```

2-2. 레포 클론

```
cd ~/workspace
git clone https://github.com/NVlabs/alpamayo.git
cd alpamayo
```

2-3. 가상환경 생성

```
uv venv --python 3.12 ar1_venv
source ar1_venv/bin/activate
```

2-4. flash-attn 이슈 해결 (의존성 설치 전에 처리)

flash-attn은 WSL2 환경에서 CUDA_HOME 미설정 등의 이유로 빌드에 실패한다. PyTorch의 SDPA(Scaled Dot Product Attention)로 대체하면 문제없이 동작한다.

`pyproject.toml` 에서 flash-attn 관련 의존성을 제거한다:

```
# pyproject.toml 열어서 dependencies 목록에서 flash-attn 항목 삭제
# 예: "flash-attn>=2.0" 같은 줄을 제거

# 또한 [tool.uv] 섹션에서 아래 줄도 함께 제거해야 한다:
# no-build-isolation-package = ["flash-attn"]
# 이 줄이 남아 있으면 uv sync 시 존재하지 않는 패키지에 대해 에러가 발생할 수 있다.
```

2-5. 의존성 설치

주의: `.venv` vs `ar1_venv` `uv sync` 는 기본적으로 프로젝트 루트의 `.venv` 디렉토리를 사용한다. 2-3에서 생성한 `ar1_venv` 를 사용하려면 반드시 `source ar1_venv/bin/activate` 로 활성화한 뒤 `uv sync --active` 플래그를 붙여야 한다. `--active` 없이 실행하면 `.venv` 에 별도로 설치되므로 주의한다.

```
uv sync --active
```

2-6. SDPA 어텐션 사용 설정

`src/alpamayo_r1/models/base_model.py` 에서 어텐션 구현을 SDPA로 변경한다. 이 프로젝트는 `AutoModelForCausalLM` 이 아닌 자체 모델 클래스를 사용하므로, 해당 파일에서 `attn_implementation` 파라미터를 찾아 수정한다:

```
# src/alpamayo_r1/models/base_model.py 내부
# attn_implementation="flash_attention_2" 를 "sdpa"로 변경
attn_implementation="sdpa"
```

참고: 실제 모델 로딩은 다음과 같은 방식으로 이루어진다:

```
from alpamayo_r1.models.alpamayo_r1 import AlpamayoR1
model = AlpamayoR1.from_pretrained("nvidia/Alpamayo-R1-10B", dtype=torch.bfloat16).to("cuda")
```

2-7. HuggingFace 토큰 로그인

nvidia/Alpamayo-R1-10B 모델에 접근하려면 HuggingFace 인증이 필요하다.

```
pip install huggingface_hub
huggingface-cli login
# 프롬프트에 HuggingFace 토큰 입력
```

2-8. 모델 다운로드

모델 크기는 약 22GB이다. 최초 실행 시 자동으로 다운로드되거나, 수동으로 미리 받아둘 수 있다.

```
# 수동 다운로드 (선택)
python -c "
from alpamayo_r1.models.alpamayo_r1 import AlpamayoR1
AlpamayoR1.from_pretrained('nvidia/Alpamayo-R1-10B')
"
```

2-9. 데이터셋 다운로드

`test_inference.py` 실행 시 추론용 데이터셋이 필요하다. 아래 HuggingFace 데이터셋에서 다운로드한다:

- <https://huggingface.co/datasets/nvidia/PhysicalAI-Autonomous-Vehicles>

이 데이터셋도 Gated 리소스이므로, HuggingFace 웹에서 접근 권한을 먼저 요청해야 한다. 모델과 마찬가지로 `huggingface-cli login` 으로 인증된 상태에서 다운로드할 수 있다.

3. 실행 방법

가상환경 활성화

```
cd ~/workspace/alpamayo
source ar1_venv/bin/activate
```

테스트 추론

```
python src/alpamayo_r1/test_inference.py
```

4. WSL2 주의사항

NVIDIA 드라이버

- WSL2 내부에 NVIDIA 드라이버를 직접 설치하면 안 된다.
- Windows 호스트에 설치된 드라이버가 WSL2로 자동 전달된다.
- `nvidia-smi` 명령으로 GPU가 정상 인식되는지 확인한다.

.wslconfig 메모리 설정

WSL2 기본 메모리 할당이 부족하면 빌드나 모델 로딩에서 실패할 수 있다. Windows 사용자 폴더에 `.wslconfig` 파일을 설정한다:

```
# C:\Users\<사용자명>\.wslconfig
[ws12]
memory=16GB
swap=8GB
```

설정 후 WSL을 재시작한다:

```
wsl --shutdown
```

CUDA_HOME 미설정 시 flash-attn 빌드 실패

WSL2에서는 CUDA_HOME 환경변수가 설정되어 있지 않은 경우가 많다. flash-attn은 빌드 시 이 변수를 요구하므로 빌드에 실패한다. 이 가이드에서는 flash-attn을 제거하고 SDPA로 대체하는 방식을 사용한다.

5. 알려진 이슈 및 해결법

flash-attn 빌드 실패

- 증상: `uv sync` 중 flash-attn 컴파일 에러 발생
- 원인: CUDA_HOME 미설정, 또는 CUDA 툴킷 버전 불일치
- 해결: pyproject.toml에서 flash-attn 의존성 제거 후 SDPA로 대체 (2-4 참조)

12GB VRAM에서 OOM (Out of Memory)

- 증상: 추론 중 `CUDA out of memory` 에러
- 해결:
 - `num_traj_samples=1` 로 설정하여 동시 처리 샘플 수를 줄인다.
 - 입력 프레임 수를 제한한다.
 - 4bit 양자화 모델 사용을 고려한다: [dwko/Alpamayo-R1-10B-4bit](#)

6. 참고 링크

- GitHub: <https://github.com/NVlabs/alpamayo>
- HuggingFace (원본 모델): <https://huggingface.co/nvidia/Alpamayo-R1-10B>
- HuggingFace (4bit 양자화): <https://huggingface.co/dwko/Alpamayo-R1-10B-4bit>
- HuggingFace (데이터셋): <https://huggingface.co/datasets/nvidia/PhysicalAI-Autonomous-Vehicles>